

Performance-Aware Leader Selection in Geo-Distributed BFT Consensus

Lucca C  zar¹[0000–1111–2222–3333], Fernando Dotti¹[1111–2222–3333–4444], and
Fernando Pedone²[2222–3333–4444–5555]

¹ PUCRS - School of Technology - PPGCC, Porto Alegre, Brazil
`lucca.dornelles99@edu.pucrs.br`, `fernando.dotti@pucrs.br`

² Universit   della Svizzera italiana, Lugano, Switzerland
`fernando.pedone@usi.ch`

Abstract. Blockchain consensus protocols have increasingly focused on performance improvements, including approaches such as better leader selection. However, the specifics of how a leader impacts consensus latency remains poorly characterized. We conduct an experimental study with geo-distributed topologies and show that the choice of leader can significantly impact consensus, affecting step durations, quorum formation, and increasing consensus time by up to 40%. Based on these findings we propose a simple, generic mechanism that uses distributed observations to steer leader selection toward higher-performing leaders. Our approach is robust to Byzantine behavior, incurs no worst-case performance overhead, and achieves throughput gains of up to 80% compared to node-removal baselines, without reducing Byzantine tolerance. While evaluated on a specific BFT protocol, the method generalizes to rotating-leader SMR systems.

Keywords: Byzantine consensus · Blockchains · Leader selection.

1 Introduction

Early blockchain platforms such as Bitcoin [8] and Ethereum [11] demonstrated fully functional permissionless distributed consensus systems, prioritizing security and correctness at the expense of performance. As modern systems experience increasing workloads, this creates a need for higher throughput and lower per-transaction latency. In response, modern research has largely shifted towards improving performance, sometimes even assuming lower security thresholds for the sake of performance [6].

Consensus algorithms are a central piece in blockchains. Several modern such algorithms employ rotating leaders or proposers, aiming to mitigate the performance impact of byzantine processes; avoid censorship; and provide fair reward distribution. While a leader coordinates the different consensus phases of a consensus instance (e.g. as in [14]), a proposer is only in charge of proposing a value for an instance and the further phases are distributed (e.g. as in [1]). In this paper we explore proposer selection as a possible avenue for performance improvement.

Although not addressed in this paper, it is expected that leader-based algorithms would be even more affected by the topology than proposer-based ones.

A *first contribution* of this paper is the set of experiments to understand the impact of the proposer’s position within the topology on the performance of geo-distributed consensus. Our experiments in Section 4 show that the position of a proposer is a significant factor throughout the entirety of consensus. In some topologies we observe overheads of up to 40% in overall consensus latency, attributed only to the position of a proposer. We also show that the position of a proposer impacts quorum formation, and therefore the duration of individual consensus steps.

The *second contribution* of this paper is, based on the above observations, the proposition of a generic proposer selection optimization mechanism. In Section 5 we define this mechanism, argue for its applicability in Byzantine environments, and evaluate it through practical experiments. Our experiments in Section 6 compare a Tendermint implementation with and without our mechanism, showing throughput improvements of up to 15% in some topologies with no impact to Byzantine tolerance.

The paper follows in Section 2 with the system model, a discussion of Tendermint and related work. It then continues with Sections 4, 5 and 6 as discussed above, followed then by Conclusions.

2 Background

2.1 System Model

We consider a distributed system composed of an unbounded set of client processes that submit transactions (or operations) to a bounded set Π of N server processes. For simplicity, here we consider that all nodes in Π act as *validators* (i.e., nodes that execute the consensus protocol). The set of nodes is dynamic and known by all nodes.

Nodes communicate by message passing. The system is partially asynchronous: it is initially asynchronous, when messages may experience arbitrarily large (but finite) delays, and there is no bound on relative process speeds. At an unknown moment (i.e., the Global Stabilization Time), becoming synchronous, when message delays and processing speed become bounded.

There are up to $f < N/3$ *faulty* or Byzantine nodes; the remaining nodes are *honest*. Faulty nodes can deviate from their specifications, whereas honest nodes adhere to them. Cryptographic techniques are used, with adversaries not having (with high probability) the computational power to subvert them.

2.2 Blockchain Consensus

We consider permissioned consensus protocols, such as PBFT [2] and Tendermint [1]. Tendermint consensus operates through three sequential phases, propose, prevote, and precommit, which functionally correspond to the pre-prepare,

prepare, and commit stages of the Practical Byzantine Fault Tolerance (PBFT) algorithm. A primary distinction of Tendermint is the implementation of a rotating proposer mechanism: by using a round-robin approach in which each node proposes only once, the network mitigates censorship [12].

Furthermore, unlike PBFT, which is permissioned, Tendermint is designed for environments where unknown participants can join. To protect against Sybil attacks [4], where a malicious actor might otherwise flood the network with nodes to gain control, the protocol integrates a Proof of Stake (PoS) mechanism. Participants receive rewards for consensus participation, while detected malicious behavior results in the destruction or reduction of their stake via slashing. This design effectively increases the cost of subversion, making it expensive for an adversary to acquire a quorum.

3 Related Work

Several works have explored performance-aware leader selection in Byzantine fault-tolerant consensus. Existing proposals differ substantially in both their objectives and integration strategies. To better position our contribution, we classify prior work along seven dimensions (see Table 1): **(I) Score Representation**, distinguishing between continuous scores and discrete node classes; **(II) Committee Eviction**, indicating whether nodes may be removed from the active committee; **(III) Evaluation Metrics**, describing the information used to estimate node quality; **(IV) Communication Overhead**, indicating whether the mechanism requires additional protocol messages or coordination phases; **(V) Decoupled Architecture**, capturing whether the optimization is implemented independently from the consensus core and can therefore be reused across different SMR protocols; **(VI) Continuous Evaluation**, indicating whether node quality is continuously updated during execution; and **(VII) Rotation Policy**, describing whether proposer rotation remains active after optimization.

Most related proposals are derived from PBFT-style primary-backup architectures and therefore focus on improving view-change behavior after detecting a slow or faulty leader (e.g., [6, 10, 16, 18]). In these systems, node differentiation primarily serves as a ranking mechanism for selecting replacement leaders after a disruption. Proposer optimization is often reactive rather than adaptive.

A second recurring characteristic is the reliance on committee eviction to sustain performance (e.g., [5, 7, 9, 13, 17]). These approaches remove nodes considered slow, unstable, or suspicious from the active consensus set. While effective at increasing throughput in controlled environments, committee reduction introduces a direct trade-off between performance and Byzantine resilience. Removing slow but correct nodes reduces the effective committee size, thereby lowering the maximum number of Byzantine faults the system can tolerate. Moreover, modern blockchain deployments already incorporate mechanisms to detect and exclude malicious validators, making additional performance-oriented eviction layers potentially redundant and operationally risky.

Algorithm	Score Type	Committee Eviction	Evaluation Metrics	Communic. Overhead	Decoupled Architecture	Continuous Evaluation	Rotation Policy
[5]	C	Y	Consensus participation and proposal success ratio	Y	Y	Y	N
[6]	C	N	Per-node consensus latency measured through active probing	Y ³	Y	N	N
[7]	D	Y	Composite reputation derived from performance and participation	Y	N	Y	N
[9]	D	Y	Latency, availability, activity level, and consensus contribution	Y	N	Y	N
[10]	D	N	Behavioral classification and response-time analysis	Y	N	Y	N
[13]	C	Y	Latency measurements combined with reputation signals	Y	N	Y	N ⁴
[15]	C	N	Observed leader throughput during evaluation windows	N	Y	N	E
[16]	C	N	Historical leader success, measured by committed blocks	Y	Y ⁵	Y	N
[17]	D	Y	Binary success/failure outcomes of consensus participation	N	N	Y	N
[18]	C	N	Consensus participation consistency, and failure frequency	Y	N	Y	N
This paper	C	N	Consensus duration	Y	Y	Y	P

Table 1: Comparison of leader selection strategies

Legend: Continuous (C) scores allow fine-grained proposer differentiation, while Discrete (D) scores classify nodes into categories. Committee Eviction indicates whether nodes may be removed from the active consensus committee. Proportional (P) rotation biases proposer frequency according to performance, whereas Equal (E) rotation preserves uniform proposer frequency among eligible nodes.

The surveyed literature also differs in how it treats proposer fairness. Many PBFT-derived optimizations effectively converge toward fixed or weakly rotating leaders, increasing susceptibility to censorship and centralization. Yu et al. [15] partially address this issue by enforcing equal rotation among nodes above a performance threshold. Our approach instead adopts proportional prioritization: higher-performing nodes are selected more frequently, while all nodes remain eligible proposers. This preserves continuous rotation while still exploiting topology-aware performance asymmetries.

Finally, existing work generally treats proposer performance as an abstract property associated with computational speed or node responsiveness, with limited investigation into how network topology shapes consensus execution. Our experiments show that proposer placement directly affects overall consensus latency, influencing the duration of subsequent protocol phases. In geo-distributed deployments, proposer location therefore becomes a first-order determinant of consensus efficiency rather than merely a secondary implementation detail.

Table 1 highlights two dominant trends in prior work: (i) performance optimization through committee reduction, and (ii) tightly coupled protocol-specific

leader management. Our approach differs in that it preserves the full committee and the original Byzantine threshold while continuously adapting proposer frequency based on observed consensus performance. Unlike eviction-based strategies, proposer prioritization improves throughput without sacrificing resilience or requiring invasive protocol modifications.

4 Understanding the Impact of Proposer Location

This section investigates how proposer’s network latency relative to other nodes affects overall consensus performance. We evaluate: (i) the measurable impact on consensus duration; (ii) effects on individual step durations; (iii) influences on quorum formation; and (iv) potential throughput gains using optimized proposer placement.

We conducted experiments on the CloudLab⁶ testbed using 13 m510⁷ nodes. Each node emulated a distinct AWS zone, with inter-node latencies⁸ injected via the `tc`⁹ (traffic control) utility to represent a realistic geo-distributed topology. The study utilized the Malachite implementation of the Tendermint protocol. Each execution lasted five minutes, including a 30-second warm-up and 15-second cool-down, with clock synchronization maintained via NTP to ensure deviations remained below 10ms.

Our experimental methodology followed a sequence of targeted tests. First, a *sanity check* was performed using a 4-node topology with a uniform 100ms latency. The results showed negligible variation ($< 1.6\%$) across proposers, both for fixed and rotating strategies, confirming that any subsequent performance discrepancies were strictly due to network topology. Also, we observed no impact in switching proposers at each decision or after a number of decisions. For better legibility, we show figures with results switching proposers every 10 decisions.

Proposer Impact on Consensus Duration. The primary metric for evaluating proposer location is the total time elapsed from the initial proposal to the final decision. In our 13-node heterogeneous topology, we observed that the physical location of the proposer is a dominant factor in system performance. As illustrated in Figure 1.a, consensus durations measured at proposers range from a minimum of 0.47s to a maximum of 0.73s depending on which node lead the round, a 40% performance gap. Figure 1.b depicts the latency from proposal to decision as measured at each node, showing that impact on termination latency at each node is also impacted by the proposer.

CDF da figura 1.a

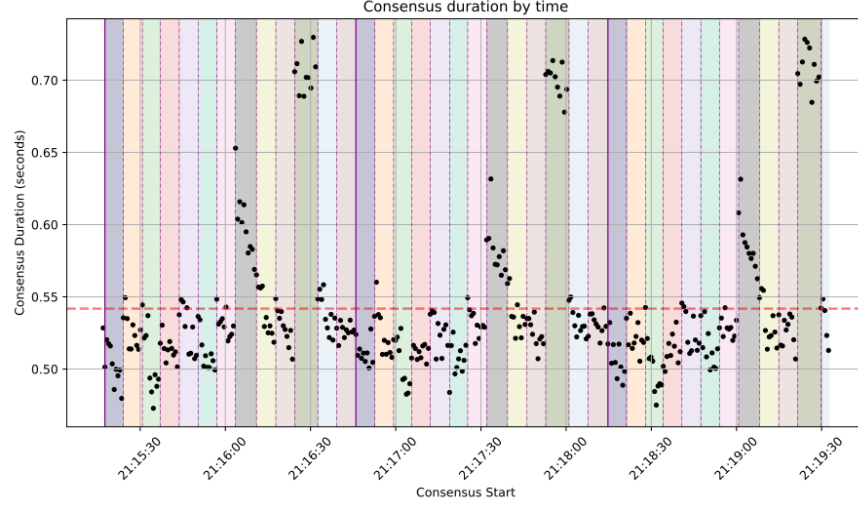
cdf da figura 1.b

⁶ <https://cloudlab.us/>

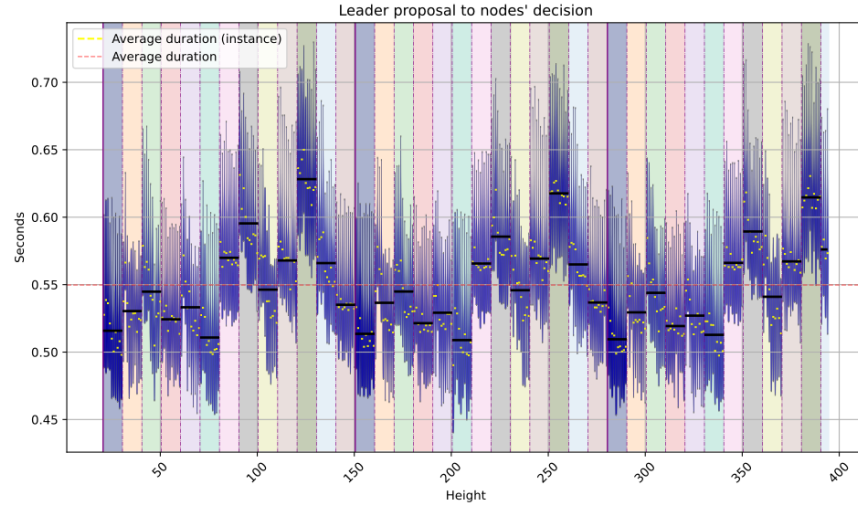
⁷ [https://docs.cloudlab.us/hardware.html#\(part.cloudlab-utah\)](https://docs.cloudlab.us/hardware.html#(part.cloudlab-utah))

⁸ <https://github.com/cometbft/cometbft/blob/v2.x/test/e2e/pkg/files/aws-latencies.csv>

⁹ <https://www.man7.org/linux/man-pages/man8/tc.8.html>



(a) Latency from propose to decide measured at each proposer



(b) Each vertical line refers to a consensus instance and is composed by plots with the consensus latency of each node

Fig. 1: (a) Consensus latency at proposer and (b) Consensus latency at all nodes. 10 instances per proposer, divided by vertical dotted lines. Solid line indicates a new round through same proposers - repeating the pattern, each proposer has a different color.

figura 2 - nao se justifica
 titulo p 4
 tail latency nos resultados

Proposer Impact on Consensus Steps. After the proposal, further communication steps are all-to-all. While the time to receive a quorum, in all-to-all communication phases, theoretically depends on the location of each receiver node, we investigated whether the proposer’s location could influence such steps. Fig.2 displays individual prevote (a) and precommit (b) step durations, showing that proposer location measurably also impacts the latency of these consensus phases.

Influences on Quorum Formation. One hypothesis to better understand the previous result is that a proposer impacts the formation of quorums, which in turn affects step durations. To check this, we computed and evaluated the quorums built at each node, rotating the whole set of proposers several rounds. We observed a 98% consistency in quorum composition when the same node acted as proposer. Nodes geographically closer to the proposer receive the proposal faster and, consequently, their votes are the first to arrive and form the *prevote* quorum. As stated in the Tendermint specification, if a farther node manages to join another node’s prevote quorum, that second node will wait for that node in the *precommit* step even if a faster quorum is reached (this is intended for Proof of Stake rewards). This means that in the precommit phase, a specific node cannot progress with it’s ideal quorum (set of closest nodes), negatively impacting latency and throughput. This also leads to the high consistency among prevote and precommit quorums in a specific node.

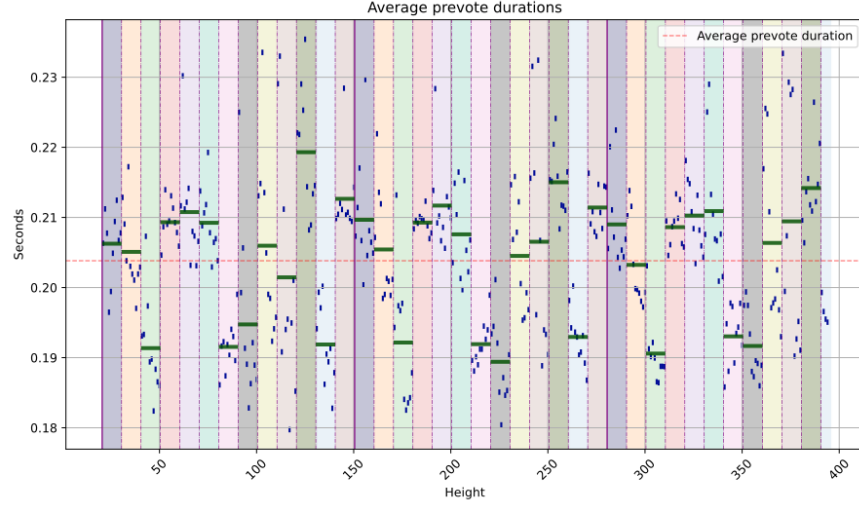
Throughput Gains via Proposer Placement The final aspect of our analysis compared the standard round-robin rotation against a fixed-proposer configuration using topologically "ideal" nodes. The results confirm that by steering the protocol to favor leaders with lower median latency to the rest of the network, the system achieves significantly higher sustained throughput.

[??? bla bla ... tem que colocar numeros aqui !!!! e fazer o texto mais sucinto]

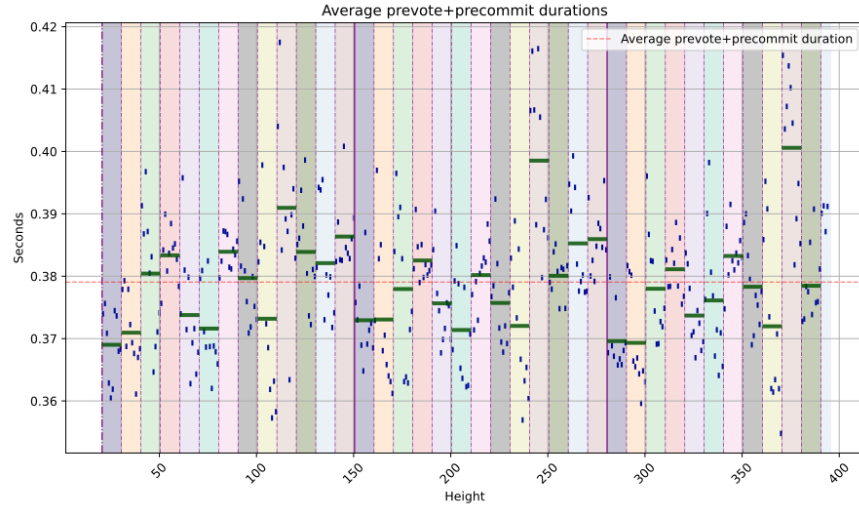
This gain is achieved without modifying the underlying BFT safety logic, proving that performance-aware leader selection is a viable optimization for geo-distributed blockchains.

5 Algorithm Proposal

As shown in the previous section, the position of a proposer does impact consensus performance. From this observation we devise an approach to dynamically adjust the frequency that each participant becomes proposer, favoring faster ones, and thus enhancing overall consensus performance. The proposed approach poses the following assumptions to the underlying consensus algorithm:



(a)



(b)

Fig. 2: (a) For each instance a we measure each nodes latencies from receiving a proposal until a quorum of prevotes is received. We then calculate the per-proposer mean (green) and global mean (red).

(b) The same for each nodes latencies from receiving the proposal until a quorum of precommits is received.

- A.1** it uses rotating proposer and provides means to record the future sequence of proposers (or parameters to calculate it);
- A.2** it has a deterministic function used at each node to calculate the next proposer. We call this function *nextProposer*. For instance, with PoS, the next proposer is a function of the known stakes from each participant.

The proposed mechanisms should ensure the following basic properties:

- P.1** the underlying consensus algorithm properties are preserved;
- P.2** disregarding stake, or with a uniform stake distribution, proposer nodes with faster (slower) decisions become proposers proportionally more (less) often;
- P.3** changes in a node's relative performance over time lead to respective adaptation of its frequency as proposer;
- P.4** every node eventually becomes proposer.

And the byzantine fault-tolerance properties:

- P.5** dishonest nodes do not impose an arbitrary sequence of proposers;
- P.6** dishonest nodes do not hinder the differentiation of faster/slower nodes.

To achieve the desired properties, three concurrent, continuous mechanisms are proposed:

- M.1** every node samples the latency and locally classifies each other node;
- M.2** every node collaborates to generate consistent global classifications;
- M.3** every node uses the global classification to calculate and adopt a new sequence of proposers.

5.1 M.1 - Sampling and classifying nodes

While the techniques for sampling and classifying nodes may vary, we describe here one with low computational cost, yet effective in our experiments. Each node continuously samples the latency of the successive consensus heights. This measure is local, comprising the time elapsed from the start of the current consensus (e.g. Tendermint's propose) to its decision (e.g. Tendermint's decided). Since each consensus height is associated to a known proposer, the sample denotes that proposer's performance.

Set of latency samples L . Let L be a set with the k last consensus latency samples taken by a node, k being a configuration parameter. At any time, the average consensus latency $\bar{L}$ is known at the node.

A parameter $T \in [0, 1]$ defines a threshold for node classification against the average latency $\bar{L}$. A latency sample s for a consensus instance can be considered *fast* if $s < \bar{L} \times (1 - T)$; *slow* if $s > \bar{L} \times (1 + T)$; and none ($\perp$) otherwise. The purpose of T is primarily to deal with short-term variations (jitter) in the observed performance of nodes, reducing the alternation of "fast" and "slow"

classifications for a same node. It achieves that by creating a neutral zone around the system average where proposal speed is considered neither "fast" nor "slow". The lower the T the more non- $\perp$ decisions are achieved, at the cost of increasing sensitivity to transient latencies. The higher the T the more resistant it is, but at the cost of less non- $\perp$ decisions.

Denote $c_h \in \{fast, \perp, slow\}$ the above classification for consensus height h .

5.2 M.2 - From local to global classifications

Now, the challenge is to derive, from these local classifications, meaningful global decisions with respect to which nodes are considered fast, slow, or none. Our algorithm is based on three steps: *vote*, *choice*, and *write*. We propose here that these steps are carried out with the underlying consensus algorithm messages and steps, having an implicit trigger associated with it. Our discussion uses Tendermint, with the following behavior at each node:

- *Vote*: upon start of height $h + 1$, send $\langle vote, h, c_h \rangle$ to all other nodes. c_h is the classification already discussed regarding consensus instance at height h . This vote is sent along with the first message for height $h + 1$ to all other nodes (e.g. Tendermint's prevote). Voting lasts until the decision for height $h + 1$. The reason for using the decision as a limit is twofold. First, the synchronization eases the integration of the mechanism and reuse of messages. And second, it allows to gather additional votes beyond the quorum needed to advance the underlying consensus phase.
- *Choice*: upon start of height $h + 2$, if a quorum of $2f + 1$ *vote* messages for a same non- $\perp$ *value* is reached, send a signed message $\langle choose, h, value \rangle_{sig}$ to all other nodes. Otherwise, send a $\langle choose, h, \perp \rangle$ non-signed message. As with the voting step, the *choose* message is sent along the first message for the next height ($h + 2$) and its reception lasts until the decision for that height.
- *Write*: Upon start of height $h + 3$, nodes gather the chosen values and decide the value to write, called $decided_h$. If a node has received a quorum of $2f + 1$ *choose* messages for a same non- $\perp$ value, the node decides that value. Otherwise decides the default value of $\perp$.

The proposer of height $h + 3$ *writes* with the proposal: the $decided_h$ value; if $decided_h \neq \perp$, the quorum of signed choices; and the updated scores of all nodes. Scores are discussed in Section 5.3. If $decided_h = \perp$, there is no need to record the quorum of signed choices. If a dishonest proposer node may omit, not writing a non- $\perp$ value, an honest next proposer may write the value with the quorum of signed choices justifying it. Each proposer node thus keeps a window of $decided_h$ with their respective set of signed choices justifying the value.

Any node, upon receiving a proposal which can be accepted by the underlying consensus algorithm, adds the following: if $decided_h = \perp$, accepts it; if $decided_h \neq \perp$, accepts if signatures are successfully verified. If not accepted, evidence of dishonest behavior is generated.

Note that all steps can happen simultaneously. During a consensus for height h , there can be an ongoing *vote* step for height $h - 1$, a *choice* step for height $h - 2$, and a *write* step for $h - 3$.

5.3 M.3 - Continuous scoring and proposer prioritization

So far we have discussed how each consensus instance h , and its associated proposer n , is associated with a classification $decided_h$ in $\{fast, \perp, slow\}$ known by all nodes. Now we address how this classification can be used to steer proposer selection. There are two main aspects to consider: (i) assuming a *nextProposer* function using the $stake_n$ of each node $n \in \Pi$ to decide the proposer sequence, how can we extend the function to prioritize faster nodes; (ii) considering that the relative performance of nodes may vary over time, due to several external factors, how the prioritization adapts to the changing relative performance of proposers.

We define $weight_n$ for each node $n \in \Pi$, as:

$$weight_n = stake_n \times [1 + (C \times normalizedScore_n)] \quad (1)$$

Instead of inputting *stake*, the *nextProposer* function is used with *weight*. C and *normalizedScore_n* are discussed next.

We define the input parameter C , the *contribution of score*, to denote how much each node's score modifies its stake. This ensures that, in systems where different stake values are possible, the score remains proportional to those different stakes. $C = 0$ disables the score algorithm and values ≥ 1 would allow nodes to be removed from the proposer pool. $0 < C < 1$ keeps all nodes in the choice and allows to differentiate them by performance. The higher the C the more performance is valued to choose the next proposer. For instance, $C = 0.3$ would limit the *weight* to $\pm 30\%$ of each node's stake, leading to a $weight_n$ in the interval of $[0.7 * stake_n, 1.3 * stake_n]$.

We now define $score_n$, α , β and M . We call $decided_n$ a value in $\{-1, 0, 1\}$ as $decide_h$ is respectively $\{slow, \perp, fast\}$. A $score_n$ of each node $n \in \Pi$ is initialized to 0. Each time a new $decided_n$ value is available, $score_n$ is updated as:

$$score'_n = trunc_{[-M..M]}(\beta * score_n + \alpha * decided_n) \quad (2)$$

β denotes how fast older decisions decay their impact on score. β is a value in $0..1$ and serves to adapt the prioritization of nodes to current observed performance. It adjusts how quickly the score returns to zero with only $\perp$ decisions.

α denotes how new decisions impact the score. α is a positive value, being a step in the modification of score. A higher α will move score faster, making score more sensitive to performance oscillations. If there are many non- $\perp$ decisions we can afford a small step, but if the number of non- $\perp$ decisions is small we are forced to have a larger α . With these definitions, for any $0 \leq \beta < 1$ score is within the interval $\pm \frac{\alpha}{1-\beta}$.¹⁰

M is a variable which informs the limits of score, which is restricted to the interval $[-M, M]$. If $M = \frac{\alpha}{1-\beta}$ it allows the full variation of scores (truncation does not take place).

We now define *nomalizedScore_n*, resulting in a value in $-1..1$, to be used in Equation 1, as:

$$\text{nomalizedScore}_n = \frac{\text{score}_n}{M} \quad (3)$$

As mentioned, the *nextProposer* function needs to be deterministic in a decentralized environment. Therefore, variables α , β , M , C , and T must be consistent across nodes. If the consensus supports self-configuration it is also possible to self-configure the global variables according to a pre-established set of rules, e.g. increasing the T if there's too much jitter and decreasing it if there's too few decisions. For systems with self-configuration, modified variables must be stored (e.g. inside a blockchain's block) so that new nodes can update their variables to match the system.

5.4 Correctness and Byzantine Fault Tolerance

The proposed mechanisms should ensure the properties described in the introduction of Section 5. Each one discussed next.

Argument for P.1: Regarding the properties of the underlying consensus algorithm, notice that the proposed mechanisms do not modify the algorithm. Messages and the value proposed are conservatively extended with votes and scores, these being processed by mechanisms separated from the underlying algorithm. The proposed mechanisms also do not prevent the progress of the underlying algorithm: whenever phase or height change may occur, the proposed mechanisms proceed with the values computed so far.

The only aspect impacted is the choice of next proposer. The structure of the function to choose the next proposer is preserved, however being parameterized with weighted stakes according to respective nodes performance, instead of stakes only. As the weighted scheme, by definition, does not nullify any stake, every node is eventually chosen as proposer.

¹⁰ Derivation of this expression omitted due to size restrictions.

Argument for P.2: As discussed in Section 5.3 the $score_n$ of a node n increases each time it is voted *fast*, leading to a multiplication of $stake_n$ by a factor > 1 , that then results in proportionally augmented $weight_n$. The weights of all nodes then substitute their original stakes in the *nextProposer* function, leading to a more frequent choice of nodes according to their weight.

Argument for P.3: The proposed mechanisms continuously sample consensus latencies, adjusting scores. The β parameter serves to decay the importance of old samples in the current score while α denotes the impact of new decisions. With this, scores adapt to changing performance observations, leading to the desired impact in the *nextProposer* choice.

Argument for P.4: This property derives from assumption A.1 and A.2 on the *nextProposer* function, implying that nodes are chosen as proposers proportionally to their stake, and the fact that the scoring mechanism does not annulate any stake, but differentiates them according to the node's performance.

Argument for P.5: A non- $\perp$ value is written if $2f + 1$ nodes choose that value. Among those, at least $f + 1$, a majority, is comprised by honest nodes, by definition. This means that dishonest nodes cannot impose a non- $\perp$ value not supported by a majority of honest ones.

Argument for P.6: Since there are at least $2f + 1$ honest nodes, if they choose the same non- $\perp$ value, then this value is eventually written by an honest proposer irrespective of dishonest nodes choices or dishonest proposers omitting the value.

Impact of dishonest behavior while voting: Properties P.5 and P.6 leave room for dishonest nodes impacting the decision if the number of honest nodes choices hc for a same non- $\perp$ value is $f + 1 \leq hc < 2f + 1$.

However, notice that dishonest nodes choices are limited: A node only chooses a non- $\perp$ value in the *choice* step if it received a quorum of that value in the *vote* step. As any two quorums have an overlap of at least one honest node, it is impossible for there to be two conflicting quorums. At the end of this step each node will either have a quorum of the same non- $\perp$ value, or default to $\perp$ if a quorum has not been received. In the first case, a dishonest node sending a non- $\perp$ choice contradicting the quorum value would immediately generate evidence of dishonest behavior. This means that colluding dishonest nodes may either add choices to a majority of honest nodes to accept their non- $\perp$ value, or not, keeping the $\perp$ decision. Keeping the $\perp$ decision does not harm performance, while accepting the non- $\perp$ value would enhance the performance observed by a majority of honest nodes.

Impact of dishonest behavior while proposing: In the role of proposer, a written non- $\perp$ value is accompanied by the signed choices justifying the quorum. Any modification of the agreed value would generate clear evidence of misbehavior

by the proposer as different signed choices from a same dishonest node would be available at honest nodes.

A dishonest proposer may write $\perp$ (which does not need the accompanying signed choices), omitting a quorum of a non- $\perp$ value. However, any honest next proposer can write the omitted non- $\perp$ value and the respective signed choices, proving the value. In practice, the omission by the dishonest node leads just to postponing the adoption of the value.

6 Experimental Evaluation

To validate the proposed mechanism, we utilized an oracle-based experimental methodology. We first executed the Malachite protocol using standard round-robin rotation to establish a baseline. From the resulting logs, we simulated the score-calculation algorithm to generate an optimized proposer sequence. Finally, we re-executed the protocol using this new sequence to measure performance gains. This iterative process was repeated up to six times (*alg-1* through *alg-6*) to observe score stabilization.

The scoring algorithm used consistent parameters across all tests: uniform node stake, a score range $M = 5$, a 50% consensus threshold ($T = 0.5$), $\alpha = 3$, $\beta = 0.9$, and a 100% contribution factor ($C = 1$). These values were chosen to prioritize rapid convergence in stable network conditions.

We repeated the experimentation with 3 different topologies, respectively cases A, B and C.

Case A. In the 13-node AWS topology, we evaluated the iterative convergence of the algorithm (*alg*) against the default rotative system (*def*). As shown in **Fig. 3**, which tracks block times across sequential runs, the high α value allows the scores to stabilize within four iterations. Once stabilized, the algorithm achieves an 8% performance gain primarily by reducing the frequency of the two most distant nodes in the proposer pool. For comparison, a traditional node-removal strategy (*rem-2*) achieves a slightly higher 10% gain but incurs a critical cost: the two most distant nodes were excluded, reducing the committee size from $N = 13$ to $N = 11$, which lowers the Byzantine fault tolerance from $F = 4$ to $F = 3$. Our approach maintains the full committee while achieving comparable performance.

Case B. To assess robustness under variable network conditions, we utilized datasets from [3] to model two cases, one with 24-hour data and another with 16-month data. In the 14-node 24-hour topology (**Fig. 4**), the scores stabilized in only three steps, yielding a 30% improvement on block time at the 80th percentile and a 15% improvement at the median. This confirms that gains are topology-dependent and also that our mechanism helps filtering out long-tail latencies.

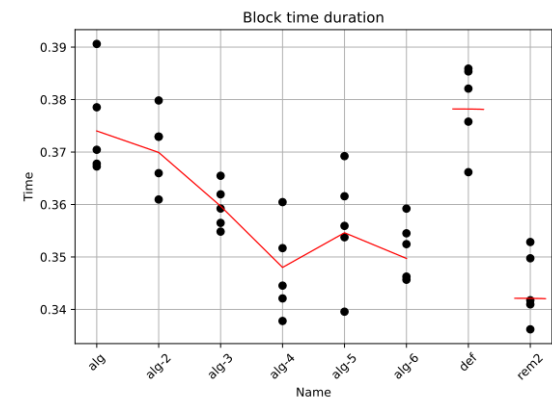


Fig. 3: Experiment with the topology used for measurements Section 3, 13 AWS zones. 5 runs (each one a plot) per configuration. Red line is the average.

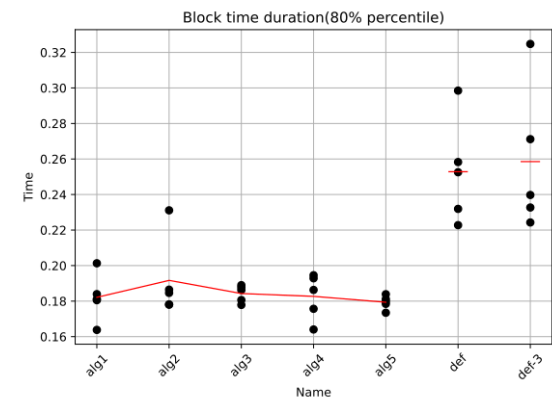


Fig. 4: Experiment with 24-hour measured 14-node topology from [3].

Case C. Finally, we evaluated a worst-case scenario using a 21-node topology modeled on 16 months of noisy data (**Fig. 5**). Despite the lack of correlation between latencies of sequential packets, and high latency variability, the algorithm remained robust. It identified subtle patterns to achieve a 6% gain at the 80th percentile and 3% at the median without degrading performance. This stability is crucial for real-world deployment, ensuring that the mechanism provides benefits when patterns exist without introducing overhead during periods of extreme instability.

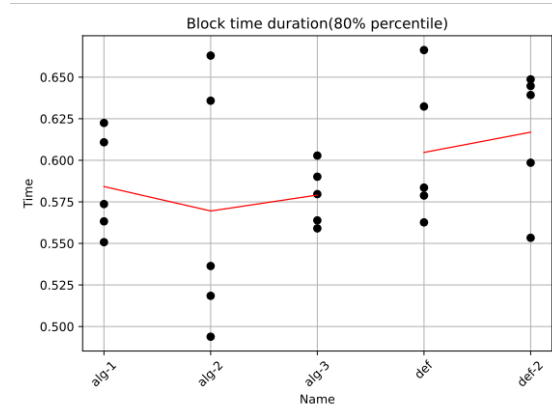


Fig. 5: Experiment with 16 months measures 21 zones topology from [3]. 5 runs per configuration with 10 rotative runs

7 Conclusion

— to be written

Note that this is an experiment with only 13 nodes, while Cosmos, a large scale implementation of Tendermint, is expected to have up to 180 nodes in the committee¹¹. It is possible that a single fixed proposer could suffer additional overhead at that scale. Nevertheless, reducing the proposer pool to a smaller subset, excluding only the most disproportionally slow nodes, could still be a realistic approach.

— In this paper we study how the position of a proposer within a topology affects distributed consensus, and propose a generic mechanism to improve performance through leader selection. As our research focuses on Tendermint, further research on other consensus algorithms is warranted to validate these findings and explore potential edge cases. In particular, algorithms such as HotStuff, with its more involved leader, are expected to have more pronounced leader impact on consensus.

¹¹ <https://docs.cosmos.network/hub/latest/validators/overview>

Finally, our mechanism is shown to improve performance with no measurable overhead. Nevertheless, several areas could be explored further, such as the impact of individual parameters and possible improvements. In particular, a more nuanced vote could lead to additional performance improvements.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Buchman, E.: Tendermint: Byzantine fault tolerance in the age of blockchains. Master's thesis, University of Guelph (2016)
2. Castro, M., Liskov, B., et al.: Practical byzantine fault tolerance. In: *OsDI*. vol. 99, pp. 173–186 (1999)
3. Di Perna, V.P., Bernardo, M., Fabris, F., Amaro, S., Matos, M., Schiavoni, V.: Impact of network topologies on blockchain performance. In: *Proceedings of the 19th ACM Int. Conf. on Distributed and Event-based Systems*. pp. 122–133 (2025)
4. Douceur, J.R.: The sybil attack. In: *International workshop on peer-to-peer systems*. pp. 251–260. Springer (2002)
5. Du, N., Liang, Z., Huang, Y., Guo, Z., Yang, H., Wang, S.: Performance optimisation method of pbft consensus for supply chain integration svm. In: *2020 7th international conference on dependable systems and their applications (DSA)*. pp. 371–377. IEEE (2020)
6. Eischer, M., Distler, T.: Latency-aware leader selection for geo-replicated byzantine fault-tolerant systems. In: *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*. pp. 140–145. IEEE (2018)
7. Huang, D., Huang, Y., Yang, Y.: Improved pbft consensus algorithm based on node evaluation and dynamic management. In: *Proceedings of the 2023 4th International Conference on Big Data Economy and Information Management*. pp. 851–855 (2023)
8. Nakamoto, S.: Bitcoin whitepaper. URL: <https://bitcoin.org/bitcoin.pdf> (17.07.2019) **9**, 15 (2008)
9. Qiao, P.: Enhancing manufacturing-as-a-service supply chain transparency with master-slave multi-chain and sqs-pbft. *IEEE Access* **12**, 186605–186616 (2024)
10. Shan, B., Gao, T., Song, L., Cui, Y.: Grouped byzantine fault-tolerant consensus mechanism based on node behaviour analysis. In: *2025 4th International Symposium on Computer Applications and Information Technology (ISCAIT)*. pp. 1890–1895. IEEE (2025)
11. Tikhomirov, S.: Ethereum: state of knowledge and research perspectives. In: *International symposium on foundations and practice of security*. pp. 206–221. Springer (2017)
12. Wahrstätter, A., Ernstberger, J., Yaish, A., Zhou, L., Qin, K., Tsuchiya, T., Steinhorst, S., Svetinovic, D., Christin, N., Barczentewicz, M., et al.: Blockchain censorship. In: *Proceedings of the ACM Web Conference 2024*. pp. 1632–1643 (2024)
13. Wen, X., Yang, X.: Mapbft: multilevel adaptive pbft algorithm based on discourse and reputation models. *The Computer Journal* **68**(6), 635–648 (2025)
14. Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I.: Hotstuff: Bft consensus with linearity and responsiveness. In: *Proceedings of the 2019 ACM symposium on principles of distributed computing*. pp. 347–356 (2019)

15. Yu, L., Wu, Y., Lu, J., Li, T.: An adaptive reputation update mechanism for primary nodes in pbft. In: 2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom). pp. 1948–1953. IEEE (2024)
16. Zhang, G., Pan, F., Tijanic, S., Jacobsen, H.A.: Prestigebft: Revolutionizing view changes in bft consensus algorithms with reputation mechanisms. In: 2024 IEEE 40th International Conference on Data Engineering (ICDE). pp. 1930–1943. IEEE (2024)
17. Zhao, F., Wang, Y.: Pbft consensus algorithm based on reward and punishment mechanism. In: 2022 3rd International Conference on Information Science and Education (ICISE-IE). pp. 168–173. IEEE (2022)
18. Zhu, X., Hu, X., Zhu, W.: Rgpbf: A reputation-based pbft algorithm with node grouping strategy. *Arabian Journal for Science and Engineering* **50**(15), 11837–11850 (2025)